Four Person Presentation Script and Rehearsal Guide

LegacyDesk attack demonstration hardening and retesting

Target running time 7 minutes | Four members | Two vulnerabilities per member

Each speaker owns the full before, fix and after sequence for their two vulnerabilities. Speak only the narration below; the action cues are for the screen operator. Use Member 1 to Member 4 until the team adds its names.

Speaker	Time window	Two vulnerabilities
Member 1	00:00 to 01:45	V01 SQL injection and V02 stored XSS
Member 2	01:45 to 03:30	V03 object access and V04 forged sessions
Member 3	03:30 to 05:15	V05 CSRF and V06 path traversal
Member 4	05:15 to 07:00	V07 misconfiguration and V08 resource consumption

How to show the fix

Prepare v1 and v2 tabs and the relevant code before recording. Show the v1 behavior, point to the implemented v2 change, then repeat the request on v2. The separate hardened version is already built; do not imply that a complete WAF, session system and proxy deployment was typed live in a few seconds. A brief code close-up makes the fix visible and reviewable.

Use the same payload and identity where applicable. V2 uses a fresh authenticated session and correct CSRF token when testing SQLi or XSS, so a CSRF failure cannot be mistaken for a WAF block. Never display the generated password file or live session tokens.

The narration contains approximately 726 spoken words. The time windows are a recording target, not a measured finished take. Rehearse the words and screen transitions together, leaving short holds so viewers can read the evidence.

Member 1 speaking script

00:00 to 01:45 | V01 SQL injection and V02 stored XSS

00 00 to 00 10

This is LegacyDesk, an employee service portal behind Nginx. We tested eight weaknesses, then created a separate hardened version. Each of us will show two attacks, their fixes, and the retests.

00 10 to 00 55

First, SQL injection. I enter this condition as the username and use a wrong password. The original application signs me in as the administrator because it joins input directly into SQL. Here is the fix in v2: a parameterized account lookup, separate password-hash verification, and validated input. The WAF adds another filter. Repeating the same payload now returns forbidden, and the direct application tests also confirm that it cannot create an administrator session.

00 55 to 01 40

Next, stored cross-site scripting. I submit this SVG comment on the original ticket. When the page renders, the event handler replaces its element with XSS executed. In v2, comments are encoded before entering HTML. CSP restricts executable content, and the WAF blocks this payload. The same public request is rejected. In the separate origin test, the payload appears as literal text and creates no SVG element. Ordinary comments still work.

01 40 to 01 45

Member two will now show object access and session security.

Screen actions

Prepare v1 login and ticket 1001, v2 HTTP evidence for V01 and V02, and the login/ticket functions side by side.

00:10: submit the SQL username with password wrong. Show administrator identity. At 00:28, point to the v2 bound query; at 00:42, show HTTP 403.

00:55: submit the SVG on v1 and hold XSS executed for 3 seconds. At 01:12, show esc(c["body"]) and CSP. At 01:28, show WAF rejection and the v2 encoded screenshot.

Fix locations: v2/app/server.py login and ticket; v2/infra/gateway.conf CSP

Member 2 speaking script

01:45 to 03:30 | V03 object access and V04 forged sessions

01 45 to 02 30

I am signed in as Alice. Changing ticket 1001 to 1002 reveals Bob payroll details in the original application. The login is valid, but permission on the selected record is missing. In v2, the shared ticket lookup checks ownership or the administrator role before reading a ticket or posting a comment. I repeat the request as Alice and receive forbidden. Alice can still open her own ticket, and the tests confirm that Bob can open his.

02 30 to 03 15

The next flaw is the identity cookie. Alice cannot open administration, but changing user_id from two to one grants access in v1. V2 replaces that integer with an unpredictable server-side session and stores only a token hash. It rotates sessions on login, expires idle sessions, and invalidates logout. Passwords are now salted hashes, and the cookie is Secure, HttpOnly and SameSite Strict. The forged original cookie now receives unauthorized; a genuine Alice session remains forbidden from administration.

03 15 to 03 30

These checks enforce two different boundaries: whether a session is valid and whether that user may access the requested record. Member three will show forged requests and file access.

Screen actions

Prepare Alice v1 ticket 1001, browser cookie tools, and V03/V04 v2 evidence. Close any administrator tab from Member 1 or use a separate Alice context.

01:45: change only id=1001 to id=1002. Hold Alice header and Bob owner together. At 02:05 show ticket_for; at 02:20 show v2 HTTP 403.

02:30: show Alice /admin denial, change user_id to 1 and reload. At 02:50 show random token and cookie flags; at 03:05 show v2 401 and Alice 403.

Fix locations: v2/app/server.py ticket_for, new_session, prepare, finish and logout

Member 3 speaking script

03:30 to 05:15 | V05 CSRF and V06 path traversal

03 30 to 04 15

Alice is signed in to the original portal. This contact-preferences form is hosted on a different origin. Submitting it changes her email because the original endpoint accepts a cookie without checking a token or Origin. V2 requires both a session-bound CSRF token and an exact allowed Origin on state-changing forms. SameSite cookies add another layer. Repeating the tokenless cross-origin request now returns forbidden, and an authenticated account read confirms that the email stayed unchanged.

04 15 to 05 00

For path traversal, I change the download filename to a parent-directory path. The original endpoint returns the private configuration fixture instead of the handbook. V2 requires authentication and accepts only an exact allowlisted filename mapped to a fixed file. It does not build a filesystem path from the request. The WAF provides another filter. The traversal request now returns forbidden; direct origin tests also reject it with not found. A normal handbook download still succeeds.

05 00 to 05 15

The CSRF fix validates the source and intent of a state change. The download fix restricts which file can be selected. Member four will finish with configuration and availability controls.

Screen actions

Prepare the v1 account, port 8091 contact form, and traversal URL. Keep v2 V05/V06 evidence ready.

03:30: submit the separate-origin form and show the changed account email. At 03:50 point to Origin and token checks; at 04:05 show HTTP 403 and unchanged-email verification.

04:15: open the traversal URL and hold the fixture configuration. At 04:32 show the exact download allowlist; at 04:47 show v2 denial and the normal handbook positive check.

Fix locations: v2/app/server.py prepare and download

Member 4 speaking script

05:15 to 07:00 | V07 misconfiguration and V08 resource consumption

05 15 to 05 55

The original public debug endpoint exposes account passwords and internal details. Its response also lacks the browser security headers we checked. V2 removes that route, uses generic errors, enforces HTTPS, and adds CSP, frame restrictions, nosniff and other policies. These headers also cover error responses, including WAF blocks. Opening debug now returns not found without account data. Generated v2 credentials remain private and are never included in our evidence.

05 55 to 06 40

Finally, repeated report generation. We use a bounded local batch of forty requests with four workers. V1 accepts all forty. V2 adds Nginx request and connection limits, atomic application quotas, cached reports, bounded workers and timeouts. The same schedule now produces five successful responses and thirty-five rate-limit responses. Allowed requests still work. Faster rejection lowers measured latency, so this is evidence of admission control, not proof of protection against every denial-of-service attack.

06 40 to 07 00

All eight attack scenarios now have before-and-after evidence. Nineteen application tests and seventeen deployed-stack checks pass. The original portal remains separate. The result is layered protection: application fixes, a WAF, traffic limits and security logs, with normal employee workflows preserved.

Screen actions

Prepare /debug on both versions, captured headers, the two load result files, and the comparison chart. Wait at least 60 seconds before rerunning a full rate-limit test.

05:15: show v1 debug briefly, then the removed route and header configuration. At 05:40 show v2 HTTP 404 and response headers.

05:55: run or play the saved 40-request result. Show 40/0 before and 5/35 after with four workers visible. At 06:20 point to Nginx limits, app quota and cache. At 06:40 show the 19-test and 17-check results.

Fix locations: v2/infra/gateway.conf; v2/app/server.py finish, report and new_session

Exact payloads and retest cues

V01 SQL injection

```
POST /login
username=' OR 1=1 --
password=wrong
```

After cue: 403 WAF block; origin test rejects login

V02 Stored cross site scripting

```
<svg onload="this.outerHTML='<p>XSS executed</p>'"></svg>
```

After cue: 403 WAF block; encoded origin output

V03 Broken object access control

```
GET /ticket?id=1002
Cookie: user_id=2
```

After cue: 403 as Alice

V04 Forged identity cookie

```
GET /admin
Cookie: user_id=1
```

After cue: 401 forged identity; 403 valid Alice

Request payloads V05 to V08

V05 Cross site request forgery

```
POST /account/email
Origin: http://127.0.0.1:8091
email=alice.updated@example.test
No CSRF token
```

After cue: 403 and unchanged email

V06 Path traversal

```
GET /download?file=../private/legacy-config.txt
```

After cue: 403 at WAF; 404 at direct origin

V07 Exposed diagnostics and missing headers

```
GET /debug
No authentication
```

After cue: 404 and secure response headers

V08 Unrestricted expensive requests

```
GET /reports/export
40 requests total
4 concurrent workers
Authenticated Alice identity
```

After cue: 40 requests and 4 workers; captured 5 HTTP 200 and 35 HTTP 429

Recording preparation

- Start and check both stacks. V1 is http://127.0.0.1:8080; v2 is https://127.0.0.1:8443. The contact form is http://127.0.0.1:8091/csrf.html. Use a local certificate exception only in the dedicated recording browser, or trust the provided local CA in that browser.
- Use separate browser contexts for v1 administrator proof, v1 Alice, and v2 Alice. Cookies share a host across ports, so verify the identity visible in each portal header before each take.
- Use alice / alice123 only for the intentionally vulnerable original. Read v2 credentials privately from v2/state/credentials.json. Do not show that file, .env, private keys, or real cookie values in the video.
- Prepare the relevant v2 functions and gateway configuration in an editor with a readable font. Keep each code close-up to the actual fix, then return to the result.
- Wait at least 60 seconds before a full quota retest. Record the status counts actually produced; a different warm quota can change the number accepted. Do not replace a live result with an invented 5/35 display.
- Restore the known proof email and remove only your own XSS or verification comments after the final take. Preserve unrelated records and the original website source.

Evidence and commands ready to use

```
python3 scripts/probe_baseline.py
python3 v2/scripts/verify_stack.py
docker compose -f v2/compose.yaml exec -T app \
  python -m unittest discover -s tests -v
```

The baseline command adds one harmless XSS comment and updates Alice email. The v2 integration command adds one ordinary verification comment. Use these commands for the full controlled run; for individual video scenes use the browser actions and exact payloads. Saved screenshots and the silent HTTP walkthrough are available if a live transition needs an edit.

Rehearsal and final recording checklist

- Rehearsal one: each member performs both cases without narration. Confirm identity, payload, before outcome, fix location, after denial, and one legitimate positive control.
- Rehearsal two: read the exact spoken script with a stopwatch. Keep the four handoffs at 01:45, 03:30, 05:15 and 07:00. Shorten pauses before cutting technical explanations.
- Rehearsal three: record one uninterrupted take. Check that payload text, HTTP status and the fix are readable, and that each speaker covers exactly two vulnerabilities.
- Record at 1080p with consistent browser zoom and microphone level. Use short silent holds on results. Hide notifications, unrelated tabs and private credentials. Put labels such as Before and After in the video overlay if useful; keep the portal itself looking normal.
- If a result differs: stop the take, check the active identity and quota window, repeat the real test, and narrate the verified result. Do not claim a patched defect solely because a WAF returned 403; show the application fix and its origin test.
- Export a video no longer than seven minutes. Watch the exported file from beginning to end, including audio, handoffs, visible evidence and final test counts. The supplied silent walkthrough is source footage, not the completed narrated team video.

What to submit

Submit the final report PDF, editable report if requested, final team recording, and private repository access as required by the faculty. Keep the team script for preparation. Replace the blank college and member details before academic submission; no personal details have been invented.